

BSc in IT: Specialising in Data Science

IT3021: Data Warehousing
and Business Intelligence

Lecture 04

Dimensional Modelling

Part 02

Content

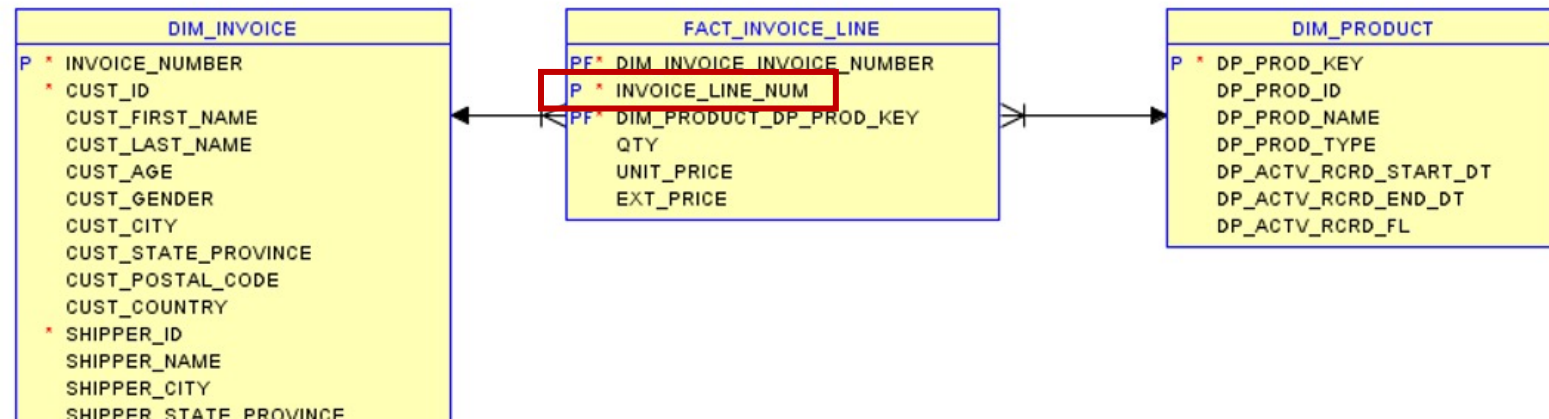
- Types of dimensions
 - Different types of dimensions
 - Slowly changing dimension types
- Types of fact tables
 - Types of measures
 - Different types of fact tables

Types of Dimensions

Degenerated Dimensions

- A dimension that has no content except for its primary key.
- Thus, dimension attribute is stored in fact table (no separate dimension table)
- e.g.,
 - *Order_No*
 - *Invoice_No*

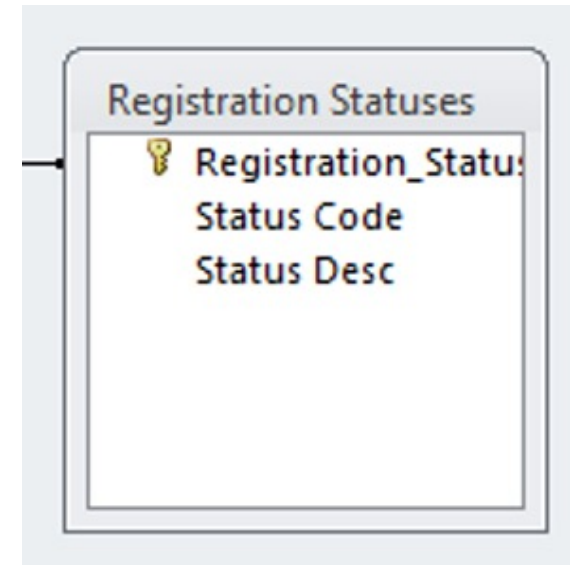
A dimension that only has an ID (no other details), so it stays directly inside the fact table.



Static Dimensions

A dimension that rarely changes, loaded manually, and doesn't need frequent updates.

- Loaded manually
- Incremental data loading (scheduled) is not required
- Rarely change
- Examples:
 - *Status_Code_Dimension*
 - Dimensions for flags/indicators

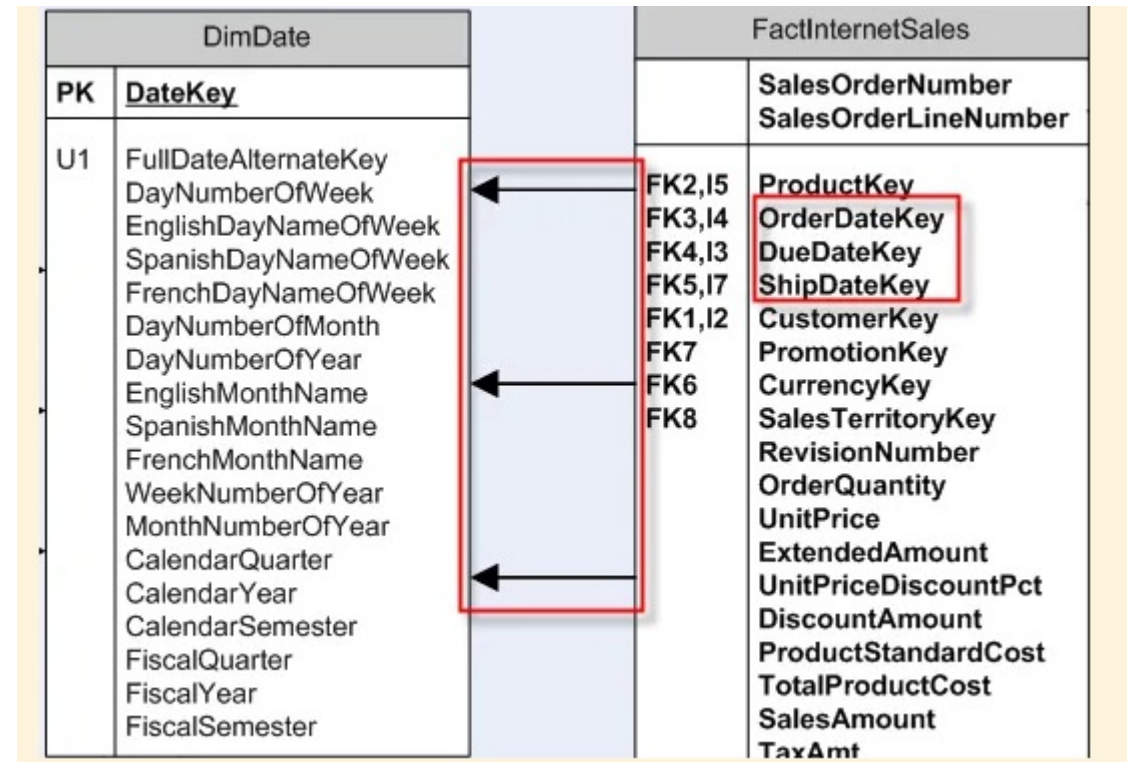


Role-Playing Dimensions

- A single physical dimension can be referenced multiple times in a fact table for multiple purposes
- e.g., *Date* dimension

A single dimension (like Date) used multiple times in the fact table for different purposes (e.g. OrderDate, ShipDate, DueDate).

FactInternetSales refers to multiple columns *DimDate*.



Date Dimension

- Attached to virtually every fact table to allow navigation of the fact table through familiar dates, months, fiscal periods, and special days on the calendar (fiscal/calendar)
- Remove the requirement to compute any date related functions, but rather want to look it up in the date dimension
- Falls into different types of dimensions static, role-playing

	DateKey	FullDateUK	FullDateUSA	DayOfMonth	DayName	DayOfWeekUK	DayOfWeekUSA	DayOfQuarter	WeekOfMonth	WeekOfQuarter	MonthName	MONTH	Quarter	YEAR	MonthYear	IsHolidayUK	HolidayUK	IsHolidayUSA	HolidayUSA
1	20130101	01/01/2013	01/01/2013	1	Tuesday	2	3	1	1	1	January	1	1	2013	Jan-2013	1	New Year's Day	1	New Year's Day
2	20130102	02/01/2013	01/02/2013	2	Wednesday	3	4	1	1	1	January	1	1	2013	Jan-2013	0	NULL	0	NULL
3	20130103	03/01/2013	01/03/2013	3	Thursday	4	5	1	1	1	January	1	1	2013	Jan-2013	0	NULL	0	NULL
4	20130104	04/01/2013	01/04/2013	4	Friday	5	6	1	1	1	January	1	1	2013	Jan-2013	0	NULL	0	NULL
5	20130105	05/01/2013	01/05/2013	5	Saturday	6	7	1	1	1	January	1	1	2013	Jan-2013	0	NULL	0	NULL
6	20130106	06/01/2013	01/06/2013	6	Sunday	7	1	1	2	1	January	1	1	2013	Jan-2013	0	NULL	0	NULL
7	20130107	07/01/2013	01/07/2013	7	Monday	1	2	1	2	1	January	1	1	2013	Jan-2013	0	NULL	0	NULL
8	20130108	08/01/2013	01/08/2013	8	Tuesday	2	3	2	2	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
9	20130109	09/01/2013	01/09/2013	9	Wednesday	3	4	2	2	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
10	20130110	10/01/2013	01/10/2013	10	Thursday	4	5	2	2	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
11	20130111	11/01/2013	01/11/2013	11	Friday	5	6	2	2	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
12	20130112	12/01/2013	01/12/2013	12	Saturday	6	7	2	2	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
13	20130113	13/01/2013	01/13/2013	13	Sunday	7	1	2	3	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
14	20130114	14/01/2013	01/14/2013	14	Monday	1	2	2	3	2	January	1	1	2013	Jan-2013	0	NULL	0	NULL
15	20130115	15/01/2013	01/15/2013	15	Tuesday	2	3	3	3	3	January	1	1	2013	Jan-2013	0	NULL	0	NULL

Range Dimensions

- Transform continuous values into categorical attributes
- Grouping of measures in a dimension
- Required for data mining algorithms where categorical values are expected
- e.g., customer age

Key	Age	Income	Purchase Count	Rating	Status	Credit Score
1	20 – 29	Less than \$20,000	0 -25	Poor	Collect	Below 500
:	:	:	:	:	:	:
27	30 – 39	\$30,000 – \$39,999	26- -50	Good	Current	750 - 774
:	:	:	:	:	:	:
74	50 – 59	Over \$100,000	150+	Excellent	Current	800+

Parent-Child Dimensions

- Based on two dimension table columns that together define the lineage relationships among the members of the dimension
- e.g., *Employee* dimension
 - *manager_ID* attribute (refers to *employee_ID*)

	EmplID	EmplName	ManagerID
▶	1	James Smith	3
	2	Amy Jones	3
	3	Paul West	Null
	4	Jill Kelley	3
	5	Jon Grande	1
	6	Jo Brown	1

Junk Dimensions

Combines small flags and indicators (like Yes/No, Status codes, Booleans) into one single dimension table, instead of creating many small dimensions.

- Transactional business processes typically produce several miscellaneous flags and indicators
- Rather than making separate dimensions for each flag and attribute, a single junk dimension can be created by combining all the flags and values together
- Need not be the cartesian product of all the attributes' possible values, but only contain the combination of values that actually occur in the fact tables
- This dimension, frequently labeled as a “*transaction profile dimension*” in a schema

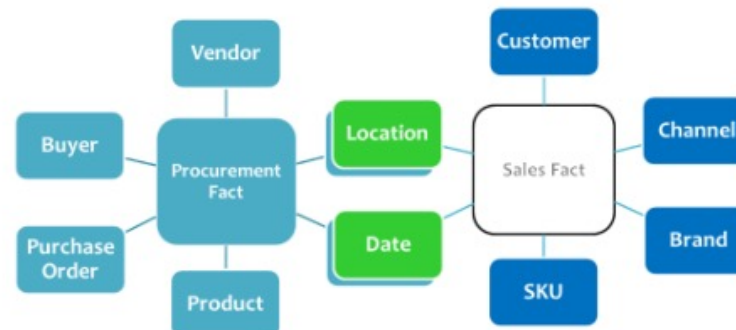
Junk Dimensions

Order Indicator Key	Payment Type Description	Payment Type Group	Order Type	Commission Credit Indicator
1	Cash	Cash	Inbound	Commissionable
2	Cash	Cash	Inbound	Non-Commissionable
3	Cash	Cash	Outbound	Commissionable
4	Cash	Cash	Outbound	Non-Commissionable
5	Visa	Credit	Inbound	Commissionable
6	Visa	Credit	Inbound	Non-Commissionable
7	Visa	Credit	Outbound	Commissionable
8	Visa	Credit	Outbound	Non-Commissionable
9	MasterCard	Credit	Inbound	Commissionable
10	MasterCard	Credit	Inbound	Non-Commissionable
11	MasterCard	Credit	Outbound	Non-Commissionable
12	MasterCard	Credit	Outbound	Commissionable

Conformed Dimensions

A dimension shared across multiple fact tables, having the same meaning and structure, ensuring consistent reporting.

- A dimension that has exactly the same meaning and content when being referred from different fact tables
 - Thus, information from separate fact tables can be combined in a single report by using conformed dimension attributes that are associated with each fact table
- For two dimension tables to be considered as conformed, they must either be identical or one must be a subset of another
 - Two dimension tables that are exactly the same except for the primary key are not considered conformed dimensions
- Conformed dimension are important in making the data warehouse being “*integrated*”
- If a particular entity had different meanings and different attributes in different source systems, there must be a single version of this entity once the data flows into the data warehouse
- Examples:
 - *Location* dimension
 - *Date* dimension



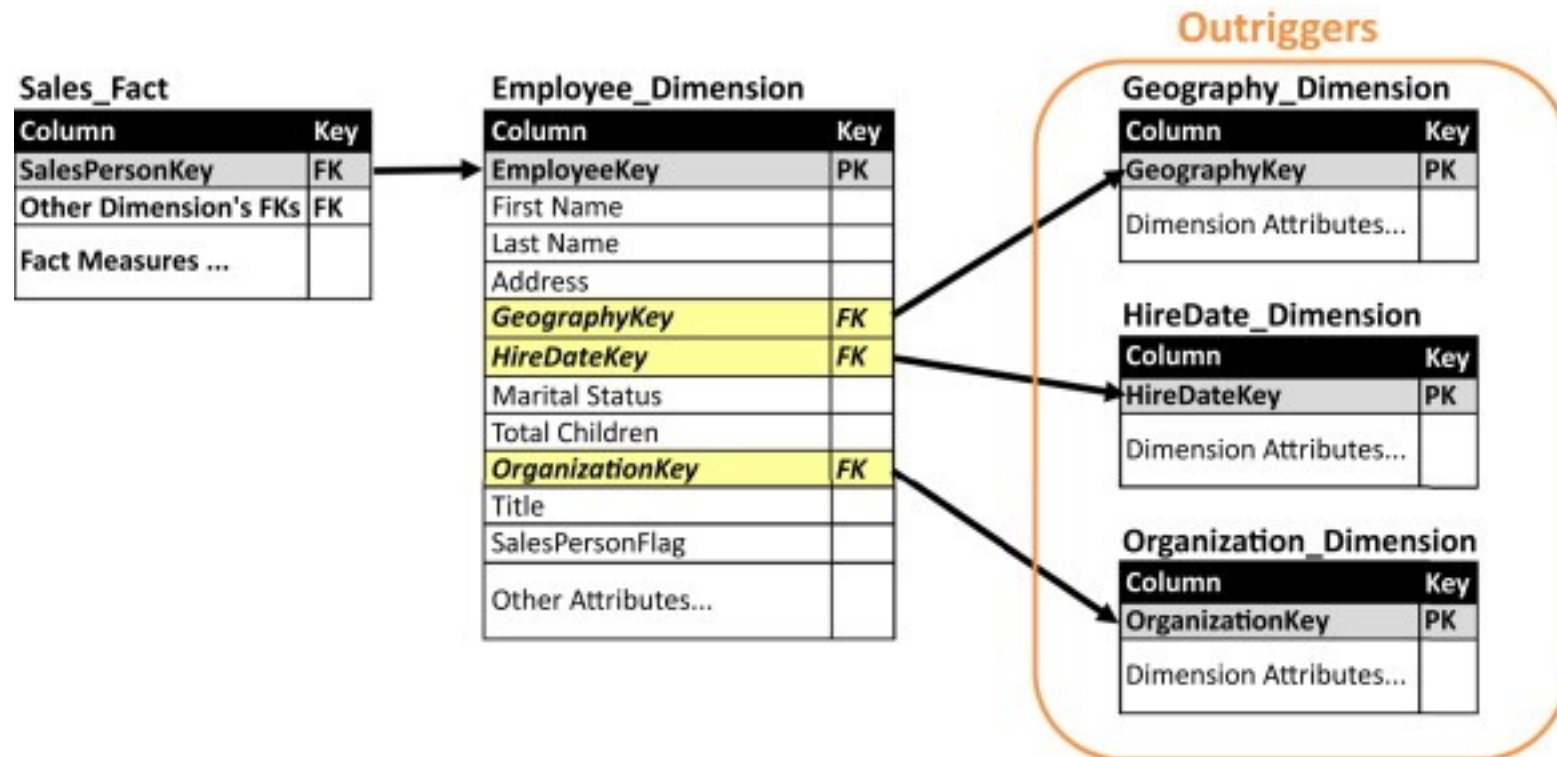
Outtrigger Dimensions

A dimension table that references another dimension table, but usually avoided to keep dimensions independent.

- A dimension can contain a reference to another dimension table
 - A bank account dimension can reference a separate dimension representing the date the account was opened
- These secondary dimension references are called outrigger dimensions
- Outtrigger dimensions are permissible, but not recommended
- In most cases, the correlations between dimensions should be demoted to a fact table, where both dimensions are represented as separate foreign keys

Outtrigger Dimensions

- *HireDate_Dimension* is as same as *Date_Dimension*???



Slowly Changing Dimensions (SCD)

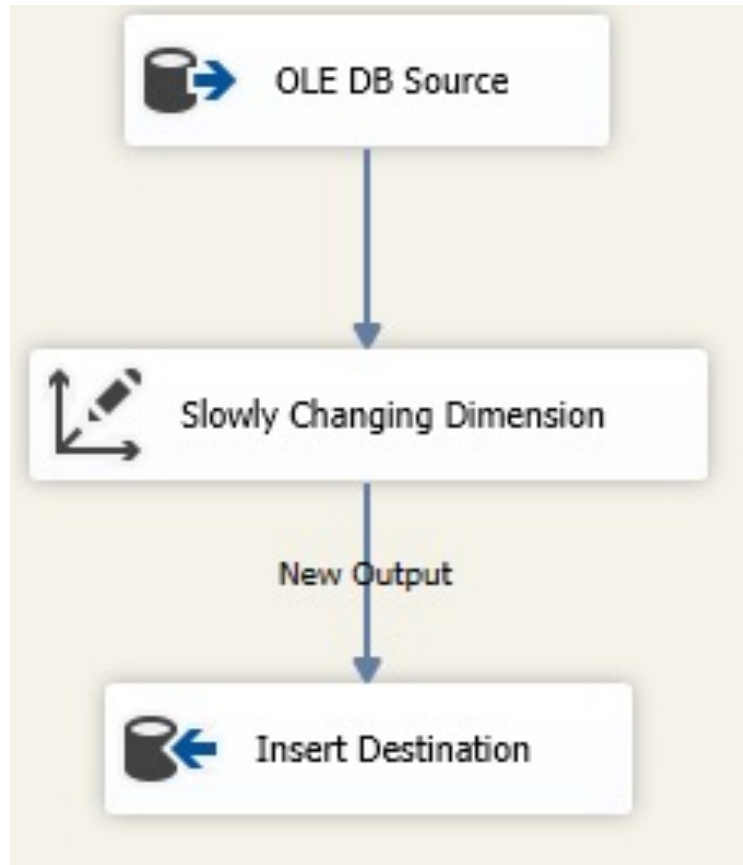
- Dimensions which contains *slowly changing dimension attributes*
- *Slowly changing dimension attributes*: an attribute of a record in a dimension table that varies over time, unpredictably
- e.g., *Customer* dimension, *Employee* dimension
- Approaches to deal with SCD attributes:
 - Type 0 – Retain original
 - Type 1 – Overwrite
 - Type 2 – Add new record
 - Type 3 – Add new attribute
 - Type 4 – Add mini-dimension
 - Type 5 – Add mini-dimension & Type 1 outrigger (hybrid)
 - Type 6 – Combine approaches of types 1,2,3 (hybrid)

Type 0 – Retain Original

Keeps the original value; no changes are made even if the attribute changes later.

- No special action is performed upon dimensional attribute value changes
- Some dimension data can remain the same as it was first time inserted, other data may be overwritten
- Appropriate for any attribute labelled “*original*”
- e.g.,
 - *customer_original_credit_score*
 - *created_date*
 - *created_by*

Type 0 – Implementation



Type 1 – Overwrite

Overwrites old data with new data, no history of changes is kept.

- Old attribute value in the dimension row is overwritten with the new value
- No history of dimension changes are tracked. Thus, is easy to maintain
- Aggregate fact tables and OLAP cubes affected by this change are recomputed
- Often use for data which changes are caused by processing corrections or for data which does not require history
- Examples:
 - Removal special characters
 - Correcting spelling errors

Type 1 – Overwrite

OLTP

CustomerID	Name	CustomerType
C0001	Cust_1	Corporate



CustomerID	Name	CustomerType
C0001	Cust_1	Retail

DW

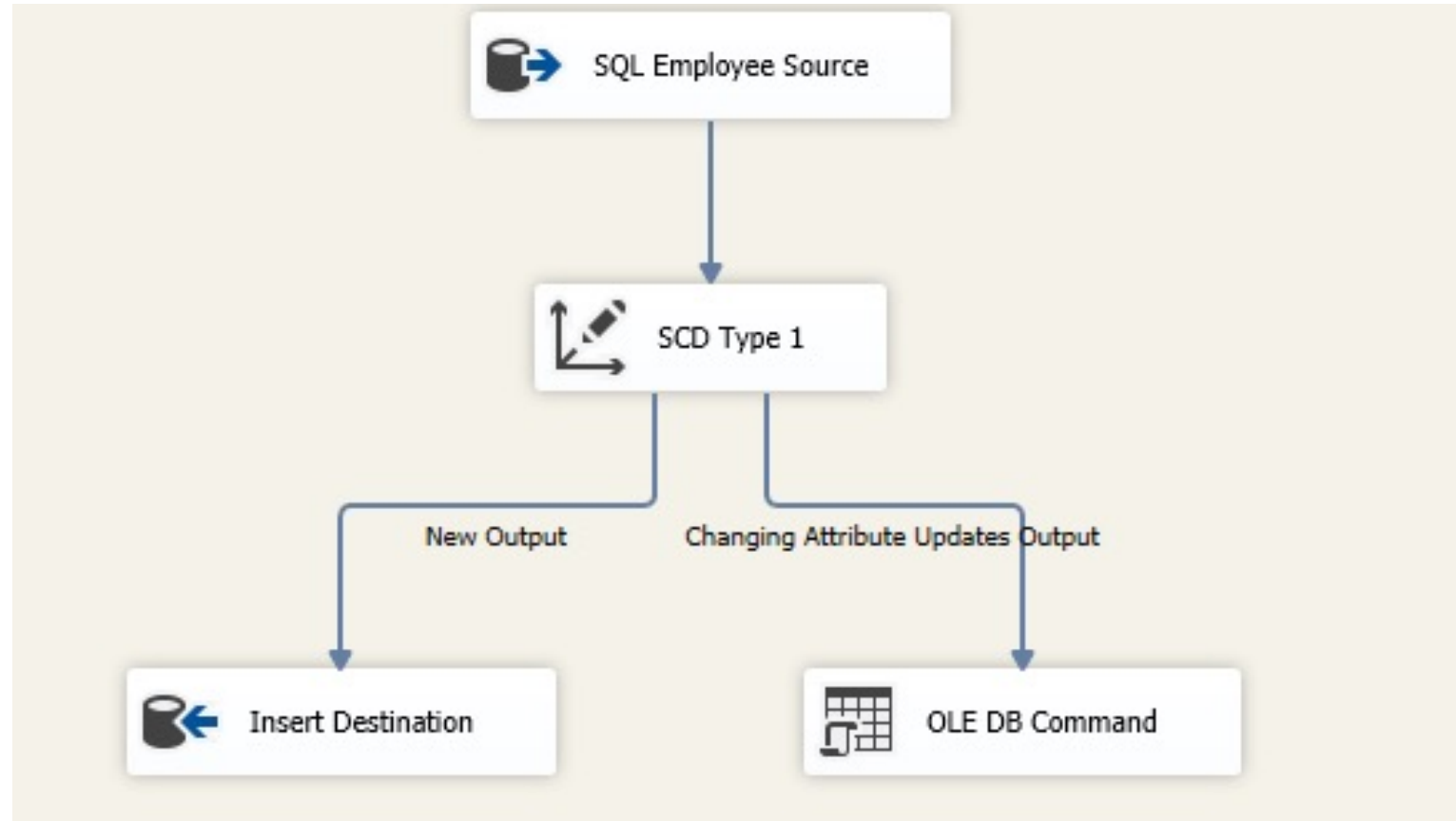
CustomerSK	CustomerID	Name	CustomerType
1	C0001	Cust_1	Corporate



CustomerSK	CustomerID	Name	CustomerType
1	C0001	Cust_1	Retail

Type 1 – Implementation

If Exists
Update
Else
Insert



Type 2 – Add New Record

- Creates a new record for a given NK with a new SK
- All history of dimension changes is kept in the database
- Also *effective_date*, *expiry_date* and *current_record_indicator* columns are used in this method
- There could be only one record with current indicator set to 'Y'
- e.g., customer address

Keeps full history by creating a new record with a new surrogate key whenever data changes.

Type 2 – Add New Record

OLTP

CustomerID	Name	Address
C0001	Cust_1	Colombo



CustomerID	Name	Address
C0001	Cust_1	Kandy

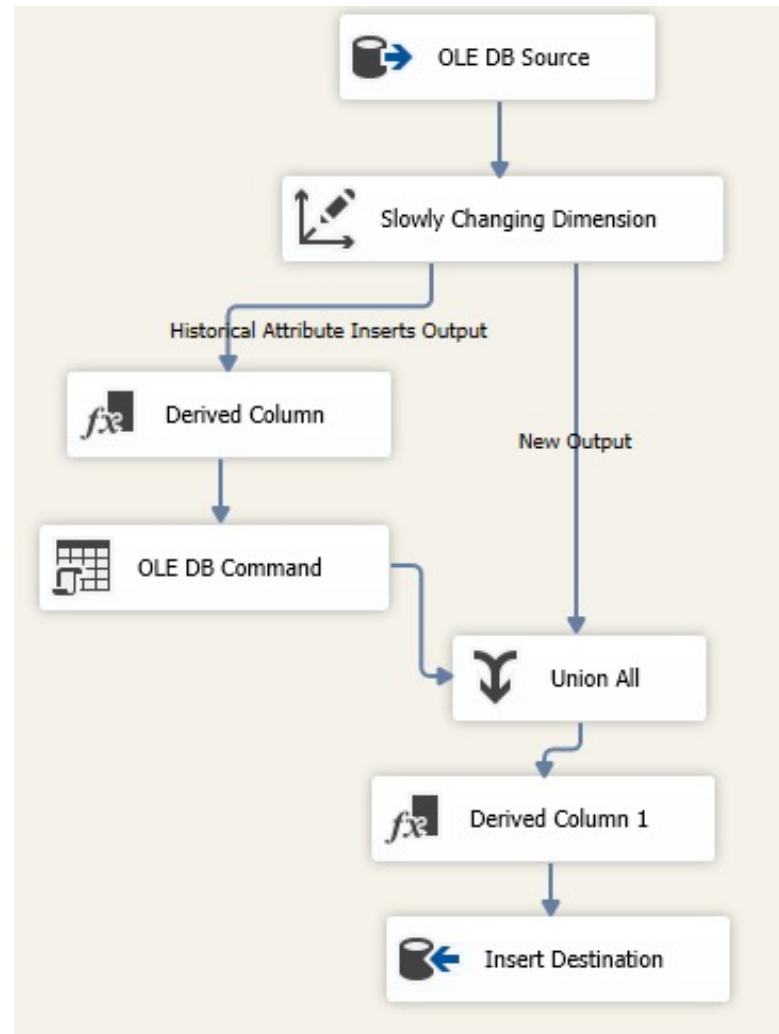
DW

CustomerSK	CustomerID	Name	Address	StartDate	EndDate	CurrentFlag
1	C0001	Cust_1	Colombo	2001-01-30	9999-12-31	Y



CustomerSK	CustomerID	Name	Address	StartDate	EndDate	CurrentFlag
1	C0001	Cust_1	Colombo	2001-01-30	2015-01-09	N
2	C0001	Cust_1	Kandy	2015-01-10	9999-12-31	Y

Type 2 – Implementation



Type 3 – Add New Attribute

- Tracks changes using separate columns
- Preserves limited history as it is limited to the number of columns designated for storing historical data
- Only the current and previous value of dimension is kept
- Used relatively infrequently

Stores limited history by adding extra columns for current and previous values.

Type 3 – Add New Attribute

OLTP

CustomerID	Name	CustomerType
C0001	Cust_1	Corporate



CustomerID	Name	CustomerType
C0001	Cust_1	Retail

DW

CustomerSK	CustomerID	Name	Current_CusType	EffectiveDate	Previous_CusType
1	C0001	Cust_1	Corporate	2001-03-31	Corporate



CustomerSK	CustomerID	Name	Current_CusType	EffectiveDate	Previous_CusType
1	C0001	Cust_1	Retail	2011-03-31	Corporate

Type 4 – Add Mini-Dimension

- A separate historical table is used to track all historical changes
- The 'main' dimension table keeps only the current data
- Mini-dimension (history table) requires its own unique primary key
- Both surrogate keys are referenced in the fact table to enhance query performance
- e.g., *customer* and *customer_history* tables

Keeps current data in main dimension and stores full history in a separate mini-dimension (history) table.

Type 4 – Add Mini-Dimension

OLTP

CustomerID	Name	CustomerType
C0001	Cust_1	Corporate



CustomerID	Name	CustomerType
C0001	Cust_1	Retail

DW

Customer

CustomerSK	CustomerID	Name	Current_CusType
1	C0001	Cust_1	Corporate



Customer_History

CustomerSK	CustomerID	Name	CusType	StartDate	EndDate
1001	C0001	Cust_1	Corporate	2001-03-31	9999-12-31

Customer

CustomerSK	CustomerID	Name	Current_CusType
2	C0001	Cust_1	Retail

Customer_History

CustomerSK	CustomerID	Name	CusType	StartDate	EndDate
1001	C0001	Cust_1	Corporate	2001-03-31	2011-03-30
1002	C0001	Cust_1	Retail	2011-03-31	9999-12-31

Type 6 – Combine Approaches of Types 1,2,3

- Add a new record as in type 2. The ***current_type*** information is overwritten with the new one as in type 1. We store the history in a ***historical_column*** as in type 3.
- Dimension table contains following additional columns:
 - ***current_type***: for keeping current value of the attribute. All history records for given item of attribute have the same current value
 - ***historical_type***: for keeping historical value of the attribute. All history records for given item of attribute could have different values
 - ***start_date***: for keeping start date of ‘*effective date*’ of attribute's history
 - ***end_date***: for keeping end date of ‘*effective date*’ of attribute's history
 - ***current_flag***: for keeping information about the most recent record

Type 6 – Combine Approaches of Types 1,2,3

OLTP

CustomerID	Name	Address
C0001	Cust_1	Colombo



CustomerID	Name	Address
C0001	Cust_1	Kandy



CustomerID	Name	Address
C0001	Cust_1	Galle

DW

CustomerSK	CustomerID	Name	Curr_Address	Hist_Address	StartDate	EndDate	CurrentFlag
1	C0001	Cust_1	Colombo	Colombo	2001-01-30	9999-12-31	Y



CustomerSK	CustomerID	Name	Curr_Address	Hist_Address	StartDate	EndDate	CurrentFlag
1	C0001	Cust_1	Kandy	Colombo	2001-01-30	2015-01-09	N
2	C0001	Cust_1	Kandy	Kandy	2015-01-10	9999-12-31	Y



CustomerSK	CustomerID	Name	Curr_Address	Hist_Address	StartDate	EndDate	CurrentFlag
1	C0001	Cust_1	Galle	Colombo	2001-01-30	2015-01-09	N
2	C0001	Cust_1	Galle	Kandy	2015-01-10	2018-03-31	N
3	C0001	Cust_1	Galle	Galle	2018-04-01	9999-12-31	Y

Types of Fact Tables

Fact Table Structure

- At the lowest grain, a fact table row corresponds to a measurement event and vice versa
 - Design of a fact table is entirely based on a physical activity and is not influenced by the eventual reports that may be produced
- Fact table contains:
 - Numeric measures
 - Foreign keys for each of its associated dimensions
 - Optional degenerate dimension keys
 - Date/time stamps

Types of Measures

- **Fully additive:** measures that can be summed across any of the dimensions associated with the fact table

Date
Store
Product
Sales_Amount

Types of Measures

- **Semi-additive:** measures that can be summed across some dimensions, but not all
 - Balance amounts are common semi-additive facts: they are additive across all dimensions except date/time
 - Another example is *inventory_level*
- **Non-additive:** measures that can not summed across any dimensions
 - e.g., ratios

Date
Account
Current_Balance
Profit_Margin

Handling Non-Additives

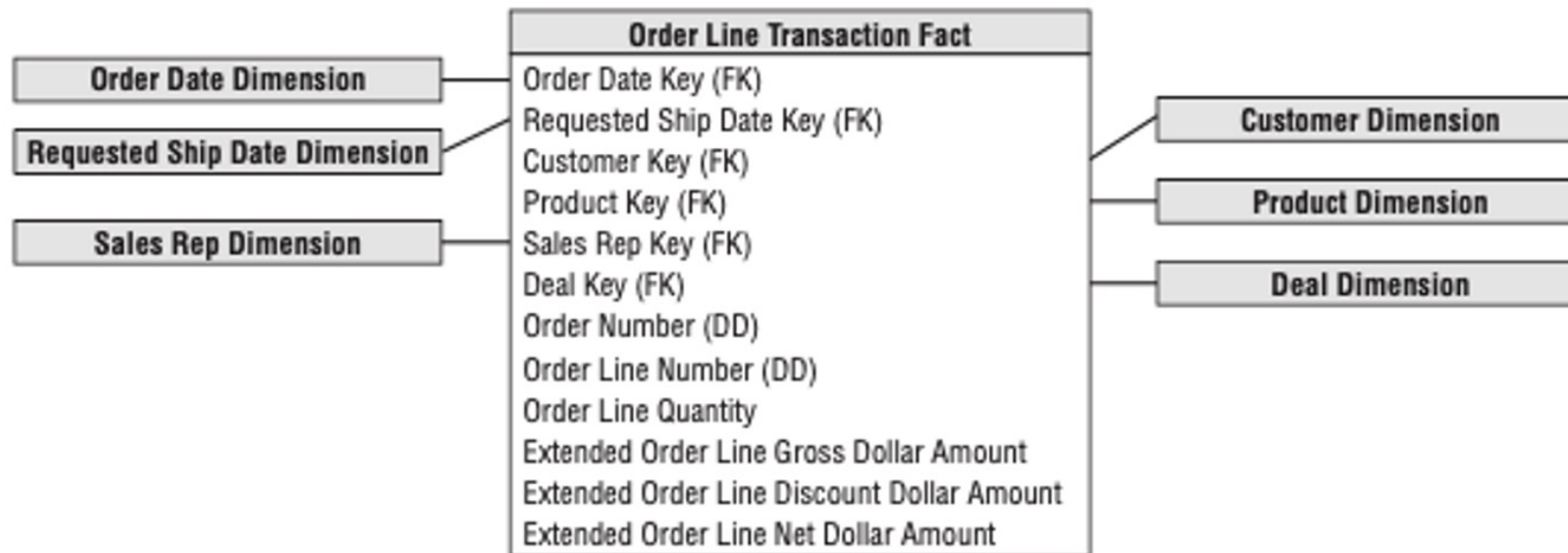
Store the additive parts first, then calculate the non-additive facts (like ratios or percentages) after aggregation.

- Whenever possible, store the fully additive components of the non-additive measure and sum these components into the final answer set before calculating the final non-additive fact.
- This final calculation is often done in the BI layer or OLAP cube
 - e.g., instead of storing the *Profit_Margin*, store *Profit* and calculate sum of *Profit_Margin* using sums of *Balances* and *Profits*

Transaction Fact Tables

Each row captures one business event at a specific time and location, with fully additive measures.

- A row in a transaction fact table corresponds to a *measurement event* at a point in space and time
 - Measures are additive

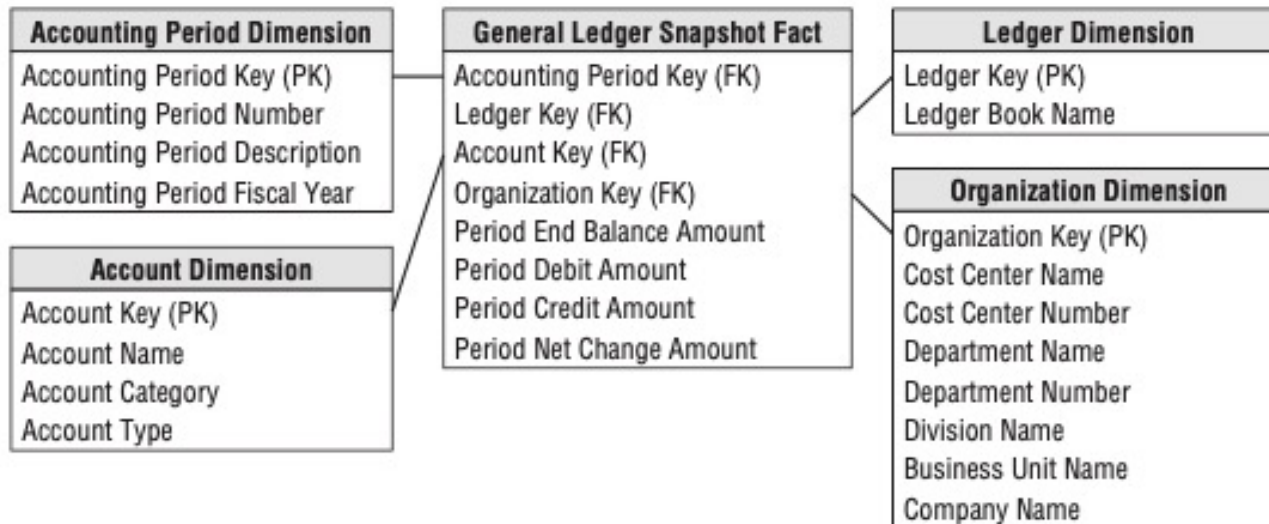


Role	Item
Foreign Key	Datetime
Foreign Key	Product
Foreign Key	Promotion
Foreign Key	Location
Degenerate	ItemID
Measure	Quantity
Measure	Tax Amount
Measure	Sale Amount
Measure	Item Total

Periodic Snapshot Fact Tables

Summarizes measurements over fixed time periods (day, week, month) rather than per transaction.

- A row in a periodic snapshot fact table summarizes many *measurement events occurring over a standard period*, such as a day, a week, or a month. The grain is the period, not the individual transaction.
- Contains semi-additive measures

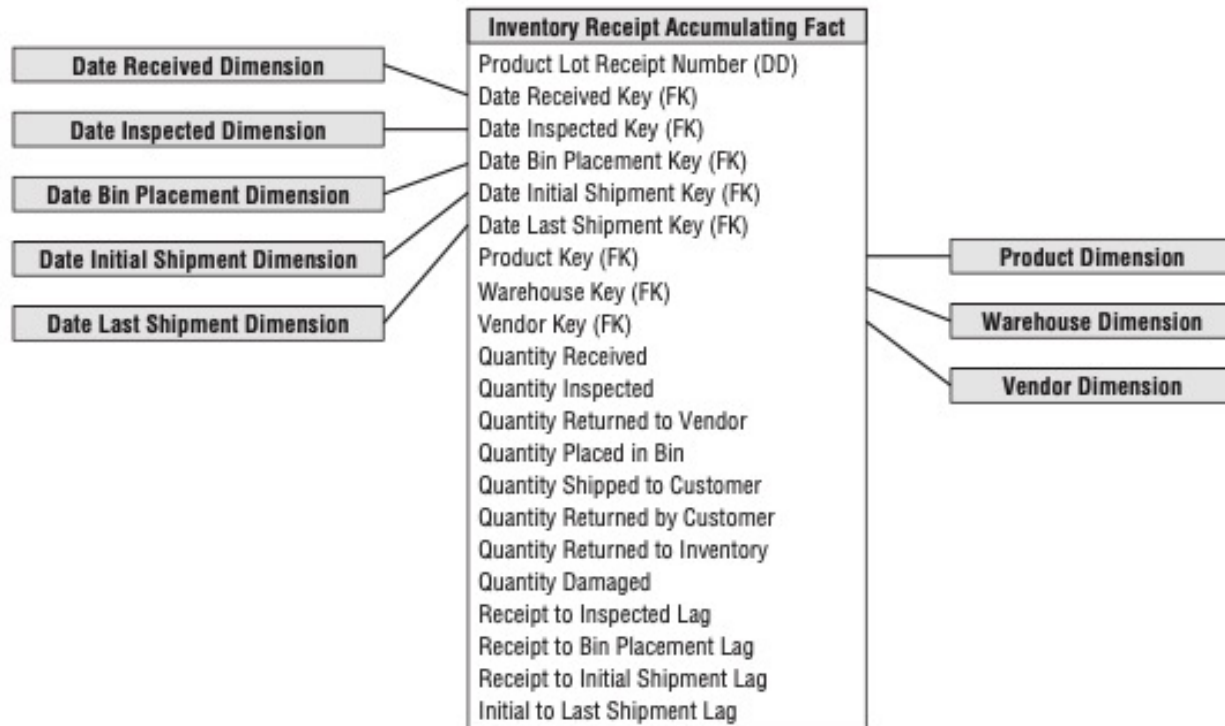


Role	Item
Foreign Key	Month
Foreign Key	Account
Foreign Key	Department
Foreign Key	Scenario
Measure	Count
Measure	Amount

Accumulating Snapshot Fact Tables

Tracks multiple stages of a process in one row, updating as each step is completed.

- A row in an accumulating snapshot fact table summarizes the *measurement events occurring at predictable steps between the beginning and the end of a process.*

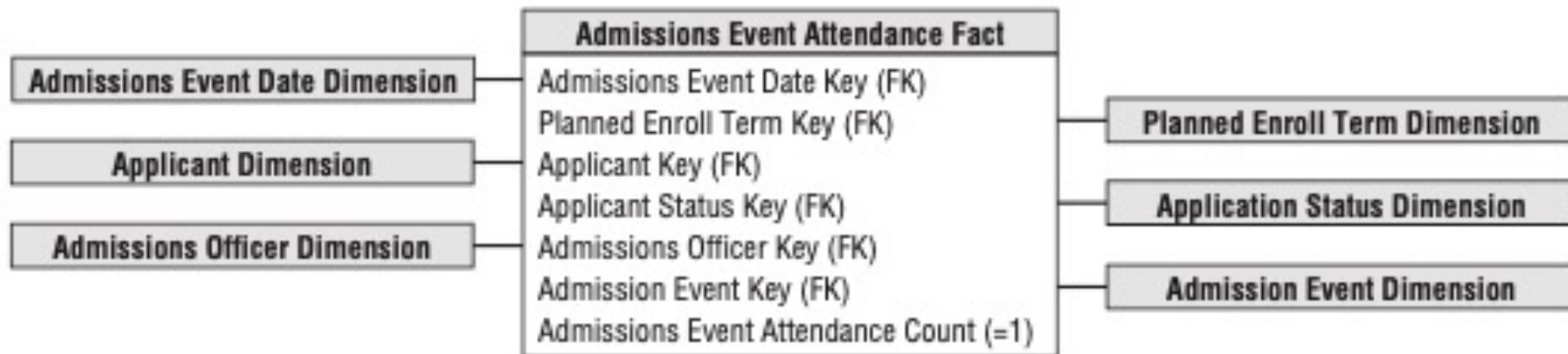


Role	Item
Foreign Key	ClaimDate
Foreign Key	PaidDate
Foreign Key	Claim
Foreign Key	Insurer
Foreign Key	Location
Measure	Reserved
Measure	Paid
Measure	Recovered
Measure	Balance

Factless Fact Tables

- It is possible that a business event merely records a set of dimensional entities coming together at a moment in time.
 - e.g., a student attending a class on a given day may not have a recorded numeric fact, but a fact row with foreign keys for calendar day, student, teacher, location, and class is well-defined.

Records events without any numeric measures, only foreign keys from related dimensions.



Aggregate Fact Tables

Stores pre-summarized data to speed up query performance by reducing the amount of data scanned.

- Aggregate fact tables are simple numeric rollups of atomic fact table data built solely to accelerate query performance.
 - Semantic layer aggregate tables
 - OLAP Cubes
 - e.g., aggregate clickstream fact table

